

Experiment 8 – SARSA and Q-Learning

NAME:N.AKSHAYA

REG.NO:192425129

COURSE CODE:MLA0305

COURSE NAME:REINFORCEMENT LEARNING

SLOT: B

8.Implementation and Comparative Analysis of SARSA and Q-Learning

Aim:

To **implement and compare SARSA and Q-Learning algorithms** in a simple Reinforcement Learning environment, evaluate their learning performance, and visualize the **reward, Q-values, and learning progress** using different colors.

Procedure:

1. Define a simple grid-world environment with states, actions, rewards, and a goal state.
2. Initialize the Q-table for both **SARSA** and **Q-Learning** with zeros.
3. Set the learning parameters:
 - Learning rate α
 - Discount factor γ
 - Exploration rate ϵ
 - Number of episodes
4. Apply the **ϵ -greedy policy** to select actions.
5. Implement SARSA using the update:
6. Implement Q-Learning using:

7. Record the total reward obtained in every episode.
8. Compare the learning performance of SARSA and Q-Learning.
9. Display summary values and visualize the results using **different colors**.

Python Code:

```
# IMPLEMENTATION AND COMPARATIVE ANALYSIS OF SARSA AND
# Q-LEARNING

import numpy as np

import matplotlib.pyplot as plt

# 1. Environment Configuration

grid_size = 5

# States: 0 to 24

# Goal = 24

goal_state = 24

# Actions

# 0 = Up

# 1 = Down

# 2 = Left

# 3 = Right

actions = [0, 1, 2, 3]

# Learning parameters

alpha = 0.1

gamma = 0.9

epsilon = 0.1

episodes = 500
```

```
max_steps = 100
```

``` # 2. Environment Function ```

```
def step(state, action):
```

```
    row = state // grid_size
```

```
    col = state % grid_size
```

```
    # Move according to action
```

```
    if action == 0:    # Up
```

```
        row = max(0, row - 1)
```

```
    elif action == 1:  # Down
```

```
        row = min(grid_size - 1, row + 1)
```

```
    elif action == 2:  # Left
```

```
        col = max(0, col - 1)
```

```
    elif action == 3:  # Right
```

```
        col = min(grid_size - 1, col + 1)
```

```
    next_state = row * grid_size + col
```

```
    # Reward system
```

```
    if next_state == goal_state:
```

```
        reward = 10
```

```
        done = True
```

```
    else:
```

```
        reward = -1
```

```
        done = False
```

```
    return next_state, reward, done
```

3. Epsilon-Greedy Policy

```
def choose_action(state, Q):  
    if np.random.rand() < epsilon:  
        # Exploration  
        return np.random.choice(actions)  
    else:  
        # Exploitation  
        return np.argmax(Q[state])
```

4. SARSA ALGORITHM

```
np.random.seed(42)  
  
Q_sarsa = np.zeros((grid_size * grid_size, len(actions)))  
  
sarsa_rewards = []  
sarsa_steps = []  
  
for episode in range(episodes):  
    state = 0  
    total_reward = 0  
    steps = 0  
    action = choose_action(state, Q_sarsa)  
    for step_count in range(max_steps):  
        next_state, reward, done = step(state, action)  
        total_reward += reward  
        steps += 1  
        if done:  
            # Terminal state
```

```

    Q_sarsa[state, action] += alpha * (
        reward - Q_sarsa[state, action]
    )
    break

# Select next action
next_action = choose_action(next_state, Q_sarsa)

# SARSA update
Q_sarsa[state, action] += alpha * (
    reward
    + gamma * Q_sarsa[next_state, next_action]
    - Q_sarsa[state, action]
)

state = next_state
action = next_action

sarsa_rewards.append(total_reward)

sarsa_steps.append(steps)

# 5. Q-LEARNING ALGORITHM
np.random.seed(42)

Q_qlearning = np.zeros((grid_size * grid_size, len(actions)))

qlearning_rewards = []

qlearning_steps = []

for episode in range(epochs):
    state = 0
    total_reward = 0

```

```

steps = 0

for step_count in range(max_steps):

    # Choose action

    action = choose_action(state, Q_qlearning)

    next_state, reward, done = step(state, action)

    total_reward += reward

    steps += 1

    # Q-Learning update

    if done:

        Q_qlearning[state, action] += alpha * (

            reward - Q_qlearning[state, action]

        )

        break

    else:

        best_next_value = np.max(Q_qlearning[next_state])

        Q_qlearning[state, action] += alpha * (

            reward

            + gamma * best_next_value

            - Q_qlearning[state, action]

        )

        state = next_state

qlearning_rewards.append(total_reward)

qlearning_steps.append(steps)

```

```
# 6. CALCULATE MOVING AVERAGE
```

```
window = 20
```

```
sarsa_average = np.convolve(
```

```
    sarsa_rewards,
```

```
    np.ones(window) / window,
```

```
    mode='valid'
```

```
)
```

```
qlearning_average = np.convolve(
```

```
    qlearning_rewards,
```

```
    np.ones(window) / window,
```

```
    mode='valid'
```

```
)
```

```
# 7. SUMMARY OUTPUT
```

```
print("=" * 60)
```

```
print("SARSA AND Q-LEARNING COMPARATIVE ANALYSIS")
```

```
print("=" * 60)
```

```
print("\nLearning Parameters")
```

```
print("-" * 60)
```

```
print("Learning Rate (Alpha) :", alpha)
```

```
print("Discount Factor (Gamma):", gamma)
```

```
print("Exploration Rate    :", epsilon)
```

```
print("Number of Episodes   :", episodes)
```

```
print("\nSARSA Results")
```

```
print("-" * 60)
```

```

print("Total Reward      :", sum(sarsa_rewards))

print("Average Reward    :", round(np.mean(sarsa_rewards), 2))

print("Best Reward       :", max(sarsa_rewards))

print("Average Steps     :", round(np.mean(sarsa_steps), 2))

print("Final Q-Value     :", round(np.max(Q_sarsa), 2))

print("\nQ-Learning Results")

print("-" * 60)

print("Total Reward      :", sum(qlearning_rewards))

print("Average Reward    :", round(np.mean(qlearning_rewards), 2))

print("Best Reward       :", max(qlearning_rewards))

print("Average Steps     :", round(np.mean(qlearning_steps), 2))

print("Final Q-Value     :", round(np.max(Q_qlearning), 2))

# 8. PERFORMANCE COMPARISON

print("\n" + "=" * 60)

print("PERFORMANCE COMPARISON")

print("=" * 60)

if np.mean(qlearning_rewards) > np.mean(sarsa_rewards):
    print("Q-Learning achieved a higher average reward.")

elif np.mean(sarsa_rewards) > np.mean(qlearning_rewards):
    print("SARSA achieved a higher average reward.")

else:
    print("Both algorithms achieved similar average rewards.")

```


9. VISUALIZATION - REWARD PER EPISODE

```
plt.figure(figsize=(10, 5))

plt.plot(

    sarsa_rewards,

    color="blue",

    alpha=0.25,

    label="SARSA Reward"

)

plt.plot(

    qlearning_rewards,

    color="red",

    alpha=0.25,

    label="Q-Learning Reward"

)

plt.xlabel("Episode")

plt.ylabel("Total Reward")

plt.title("SARSA vs Q-Learning - Reward per Episode")

plt.legend()

plt.grid(alpha=0.3)

plt.show()
```

10. VISUALIZATION - MOVING AVERAGE

```
plt.figure(figsize=(10, 5))

plt.plot(

    range(window, episodes + 1),
```

```
sarsa_average,  
color="green",  
linewidth=2,  
label="SARSA Moving Average"  
)  
  
plt.plot(  
    range(window, episodes + 1),  
    qlearning_average,  
    color="orange",  
    linewidth=2,  
    label="Q-Learning Moving Average"  
)  
  
plt.xlabel("Episode")  
plt.ylabel("Average Reward")  
plt.title("Learning Performance Comparison")  
plt.legend()  
plt.grid(alpha=0.3)  
plt.show()  
  
# 11. VISUALIZATION - Q VALUES  
  
plt.figure(figsize=(10, 5))  
  
plt.plot(  
    np.max(Q_sarsa, axis=1),  
    color="purple",  
    marker="o",
```

```

        label="SARSA"
    )
plt.plot(
    np.max(Q_qlearning, axis=1),
    color="brown",
    marker="s",
    label="Q-Learning"
)
plt.xlabel("State")
plt.ylabel("Maximum Q-Value")
plt.title("Comparison of Learned Q-Values")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```

12. BAR CHART - FINAL PERFORMANCE

```

algorithms = ["SARSA", "Q-Learning"]
average_rewards = [
    np.mean(sarsa_rewards),
    np.mean(qllearning_rewards)
]
plt.figure(figsize=(8, 5))
bars = plt.bar(
    algorithms,
    average_rewards,

```

```

    color=["blue", "red"],
    width=0.5
)

plt.ylabel("Average Reward")

plt.title("Average Reward Comparison")

# Display values on bars
for bar in bars:

    height = bar.get_height()

    plt.text(

        bar.get_x() + bar.get_width() / 2,

        height,

        f"{height:.2f}",

        ha="center",

        va="bottom",

        fontsize=11

    )

plt.grid(axis="y", alpha=0.3)

plt.show()

# 13. DISPLAY LEARNED POLICY

symbols = {

    0: "↑",

    1: "↓",

    2: "←",

    3: "→"

```

```

}

print("\nSARSA Learned Policy")

print("-" * 40)

for row in range(grid_size):\

    for col in range(grid_size):

        state = row * grid_size + col

        if state == goal_state:

            print(" G ", end=" ")

        else:

            action = np.argmax(Q_sarsa[state])

            print(

                f" {symbols[action]} ",

                end=" "

            )

    print()

print("\nQ-Learning Learned Policy")

print("-" * 40)

for row in range(grid_size):

    for col in range(grid_size):

        state = row * grid_size + col

        if state == goal_state:

            print(" G ", end=" ")

        else:

            action = np.argmax(Q_qlearning[state])

```

```

        print(
            f" {symbols[action]} ",
            end=" "
        )

    print()

    print("\n" + "=" * 60)

    print("EXPERIMENT COMPLETED SUCCESSFULLY")

    print("=" * 60)

```

Result:

SARSA and Q-Learning were successfully implemented and compared in a grid-world reinforcement learning environment. Both algorithms learned an effective policy through repeated interaction with the environment. SARSA uses the action actually selected for its update, making it an on-policy algorithm, whereas Q-Learning uses the maximum estimated future Q-value and is an off-policy algorithm. The reward curves, moving-average graph, Q-values, and comparison bar chart provide a clear visualization of their learning performance.

Summary Result:

```

=====
SARSA AND Q-LEARNING COMPARATIVE ANALYSIS
=====

Learning Parameters
-----
Learning Rate (Alpha) : 0.1
Discount Factor (Gamma): 0.9
Exploration Rate      : 0.1
Number of Episodes    : 500

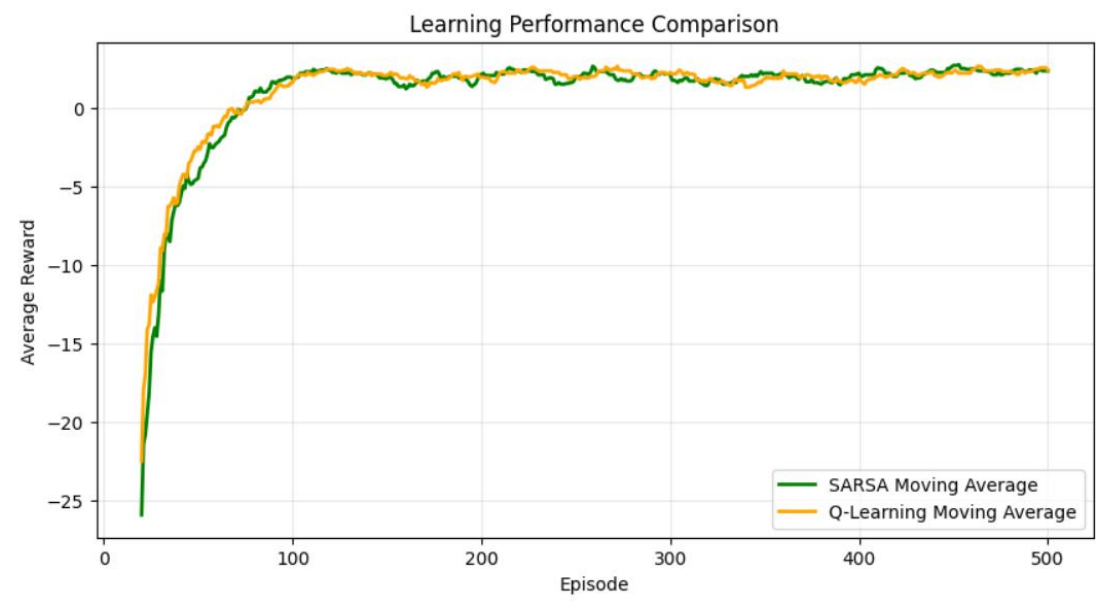
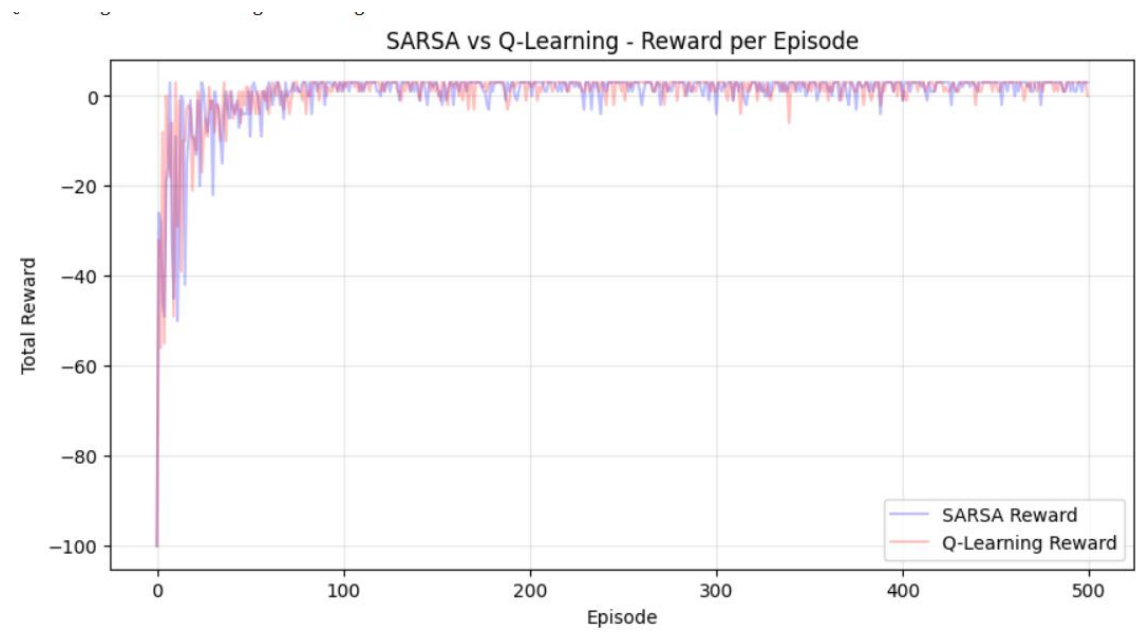
SARSA Results
-----
Total Reward          : 216
Average Reward        : 0.43
Best Reward           : 3
Average Steps         : 10.55
Final Q-Value         : 10.0

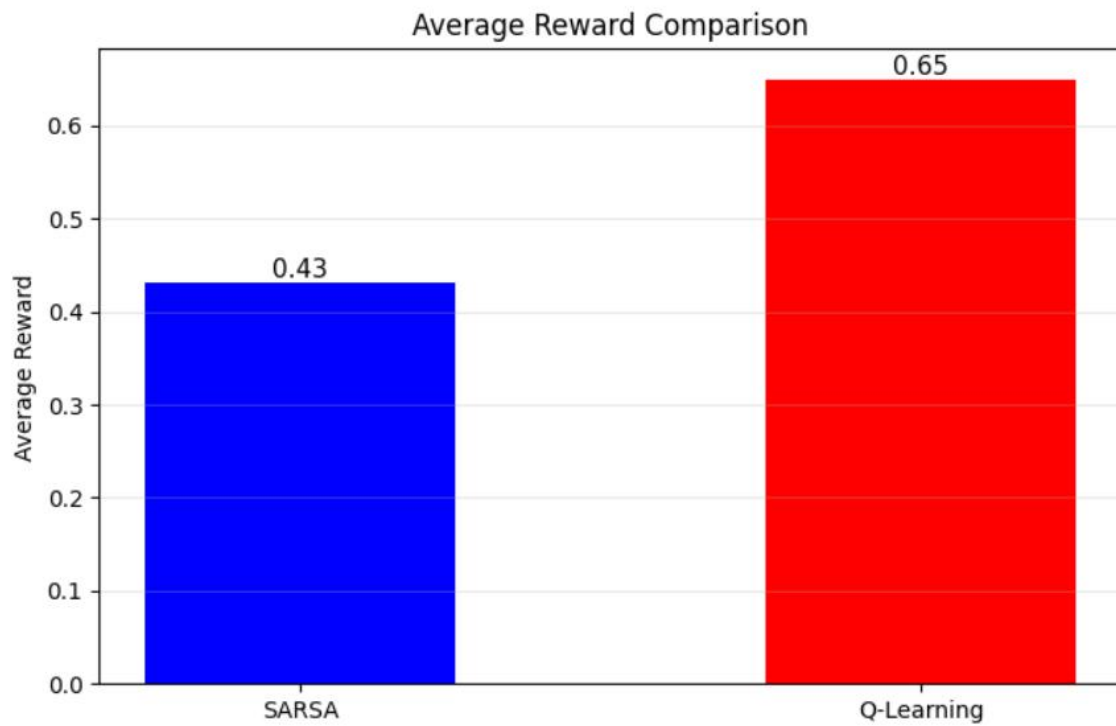
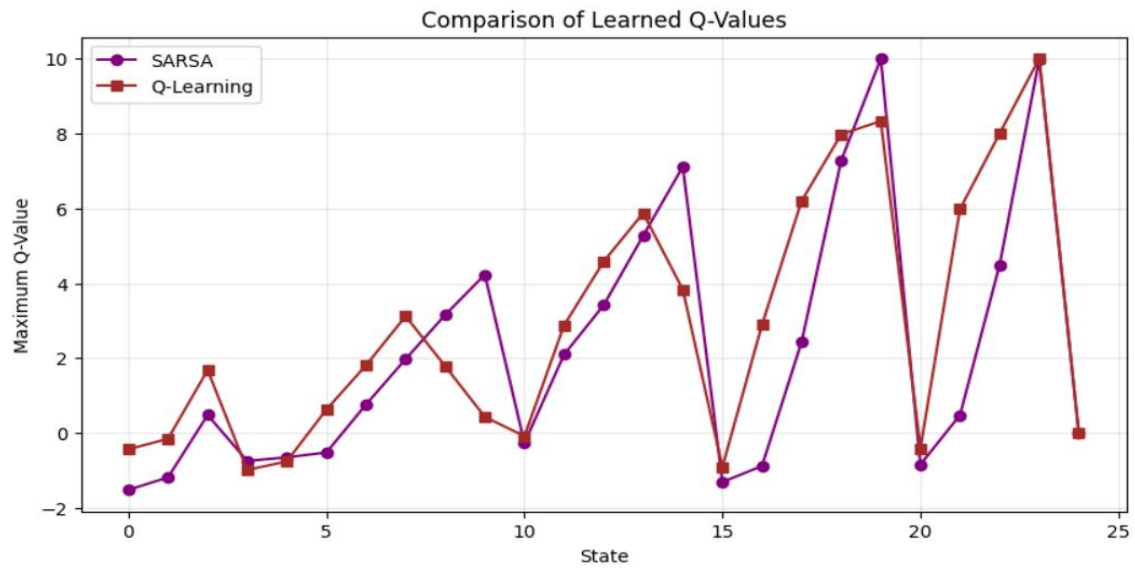
Q-Learning Results
-----
Total Reward          : 325
Average Reward        : 0.65
Best Reward           : 3
Average Steps         : 10.33
Final Q-Value         : 10.0

=====
PERFORMANCE COMPARISON
=====
Q-Learning achieved a higher average reward.

```

Visualization Result:





SARSA Learned Policy

```

↓ → ↓ ↓ ↓
→ → ↓ ↓ ↓
→ → → → ↓
→ → → ↓ ↓
→ → → → G
  
```

Q-Learning Learned Policy

```

↓ → ↓ ↓ ↓
→ → ↓ ↓ ↓
→ → ↓ ↓ ↓
→ ↓ ↓ ↓ ↓
→ → → → G
  
```

=====

EXPERIMENT COMPLETED SUCCESSFULLY

=====